ECE Paris – OMNES Education
Big Data Frameworks · ING4 Data & IA

Compte rendu – Lab dbt

Bronze → Silver avec dbt et DuckDB

Déclaration des sources bronze sur S3, modèles silver typés et dédoublonnés, tests de qualité des données, lineage, export Parquet et exercices, sur la plateforme Onyxia (Adaltas)

Auteure	Dorline TIAKO OKASSIE
Groupe	ING4 – Groupe 01 – Data & Intelligence Artificielle
Environnement	Service Onyxia `vscode-pyspark` (4 CPU / 4000 Mi), dbt Core 1.12.6, dbt-duckdb 1.11.1, DuckDB 1.5
Bucket	`s3://user-d-tiakookassie-ece`
Dépôt Git	`github.com/Dorline0121/ece-2026-bigdata` – dossier `lab_medallion/`
Date	5 novembre 2026

Sommaire

1. Objectifs du lab
2. Difficultés rencontrées et contournements
3. Environnement, installation et projet dbt
4. Couche bronze : déclaration des sources
5. Couche silver : modèles `stg_users` et `stg_orders`
6. Tests de qualité des données
7. Pipeline complet, lineage et documentation
8. Export de la couche silver en Parquet
9. Exercices
10. Synthèse des résultats
11. Conclusion

1. Objectifs du lab

- Créer un projet dbt utilisant DuckDB comme moteur d'exécution.
- Déclarer les fichiers CSV stockés sur S3 comme **sources** de la couche bronze.
- Construire la couche **silver** : des modèles typés, nettoyés et dédoublonnés.
- Valider les données avec des tests génériques et singuliers, et analyser les échecs.
- Exécuter tout le pipeline avec `dbt build` et explorer son graphe de dépendances.

L'architecture visée est la suivante : dbt ne traite pas les données lui-même, il compile du SQL et l'envoie à DuckDB, qui lit les fichiers bruts sur S3.

```
S3 bronze/                          dbt (compile le SQL)
 ├── users.csv  ──→ stg_users           │
 └── orders.csv ──→ stg_orders          ▼
                      Silver       DuckDB  ──lit──→  S3 bronze/*.csv
```

2. Difficultés rencontrées et contournements

Plusieurs problèmes, liés à la plateforme ou au sujet, ont dû être résolus avant ou pendant le lab.

① Quota Onyxia bloqué par des services fantômes

Constat. Les identifiants S3 temporaires de l'ancien service avaient expiré (`InvalidAccessKeyId`), il fallait donc lancer un service neuf. Mais le quota restait bloqué à 100 % (CPU 4/4, RAM 4000Mi/4Gi, `Count/pods = 2`) alors qu'aucun service n'apparaissait. Deux anciens StatefulSets (`vscode-pyspark-836949` et `-351192`), dont les volumes n'existaient plus, réservaient encore les ressources.

Contournement. Sans accès `kubectl`, la suppression a été demandée à l'équipe pédagogique, qui a libéré le quota. Un service neuf a ensuite été lancé avec 4000m de CPU et 4000Mi de mémoire, et le dépôt Git cloné automatiquement.

② Profil AWS `default` absent du nouveau service

Constat. `aws --profile default` échouait (`The config profile (default) could not be found`) : ce profil avait été créé dans l'ancien service, au lab S3.

Contournement. Il a été recréé à partir des variables d'environnement fournies par Onyxia (`aws configure set ... --profile default`, avec l'endpoint SeaweedFS et le style d'URL `path`).

③ Modèle `states` référencé mais jamais créé par le sujet

Constat. Le fichier de tests du sujet contient un test `relationships` vers `ref('states')`, mais aucun modèle de ce nom n'est créé.

Contournement. Création d'un **seed** `seeds/states.csv` de 62 codes : 50 États, DC, 8 territoires et les 3 codes militaires (`AA`, `AE`, `AP`), qu'utilise le générateur d'adresses.

④ Seeds CSV exclus de Git par la règle `*.csv`

Constat. La règle `*.csv` du `.gitignore`, ajoutée au lab DuckDB pour ne pas versionner les gros jeux de données, excluait aussi les seeds, qui sont pourtant du code source du projet.

Contournement. Ajout forcé des seeds avec `git add -f seeds/states.csv seeds/zip_prefixes.csv`.

⑤ Autres ajustements

- **Authentification GitHub** : le service n'ayant pas de token, le premier `git push` a été refusé (`No anonymous write access`). La connexion a été faite par le *device login* de VS Code.
- **Collage partiel des blocs *heredoc*** : deux fois, le fichier n'a pas été créé et dbt a répondu `Nothing to do` (figures 12 et 17). Il a fallu recoller le bloc en entier.
- **Suppression de lignes avec `sed '1,2d'`** : elle a abîmé le test singulier, qui a dû être réécrit (exercice 3).
- **`uv run duckdb`** : le paquet Python ne fournit pas de CLI, c'est le `duckdb` du système qui a été utilisé.
- **`dbt build` sans `--select silver`** : sur une base vide, le seed `states` doit être rechargé avant le test `relationships` qui en dépend.

3. Environnement, installation et projet dbt

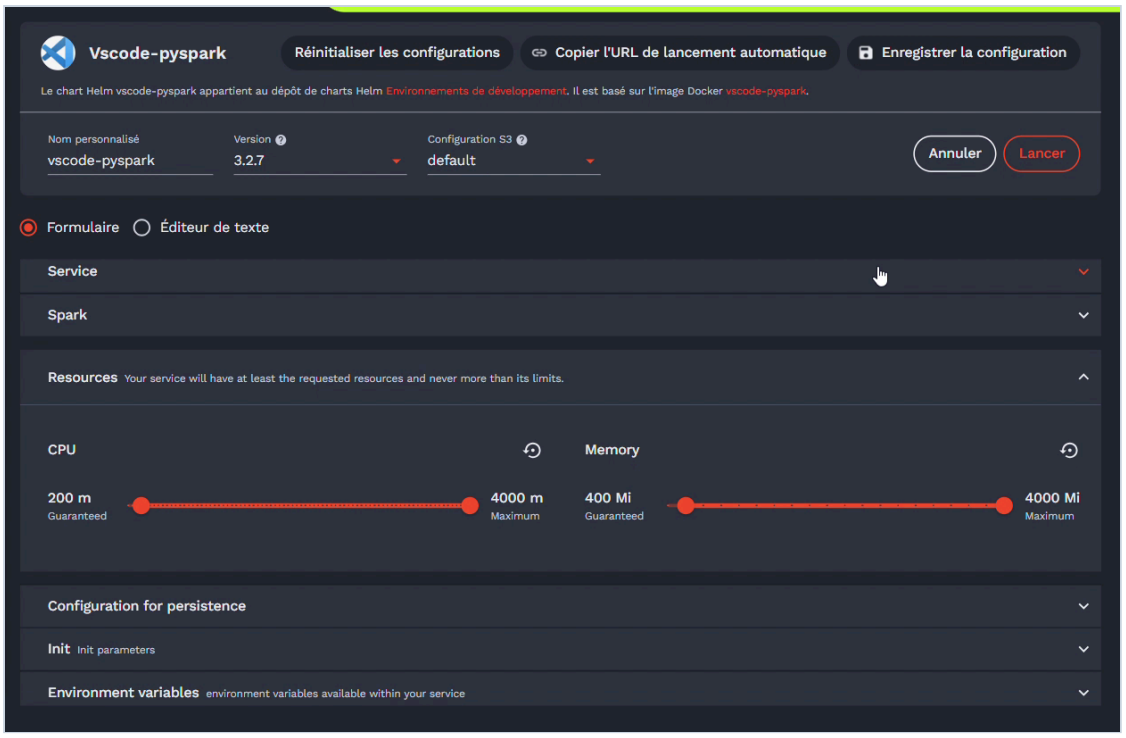


Figure 1 – Configuration du nouveau service Onyxia : limites à 4000 m de CPU et 4000 Mi de mémoire

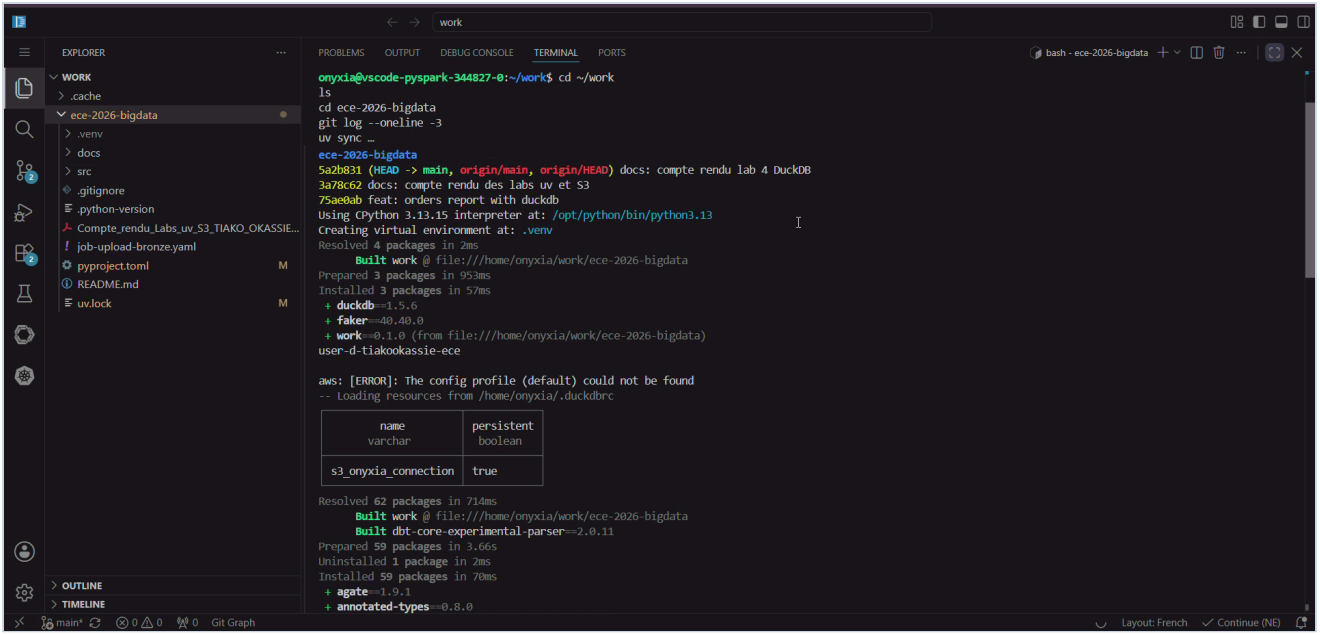


Figure 2 – Dépôt à jour (5a2b831 en tête), environnement recréé avec uv sync, secret DuckDB présent, mais profil AWS default manquant (difficulté ②)

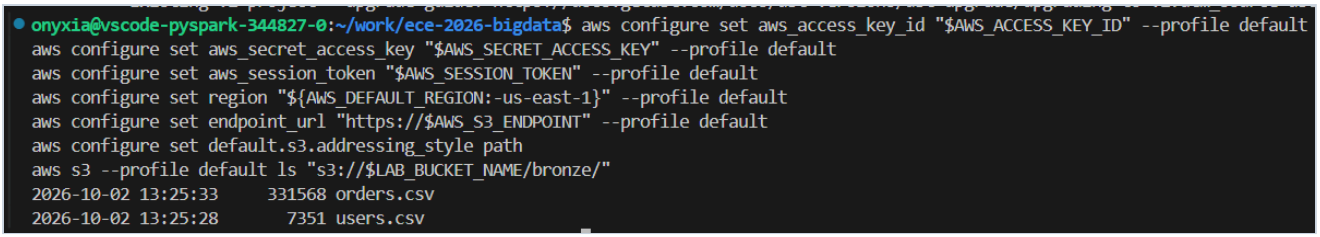


Figure 3 – Profil AWS default recréé : la couche bronze est de nouveau accessible

```
uv add dbt-duckdb
uv run dbt --version
```

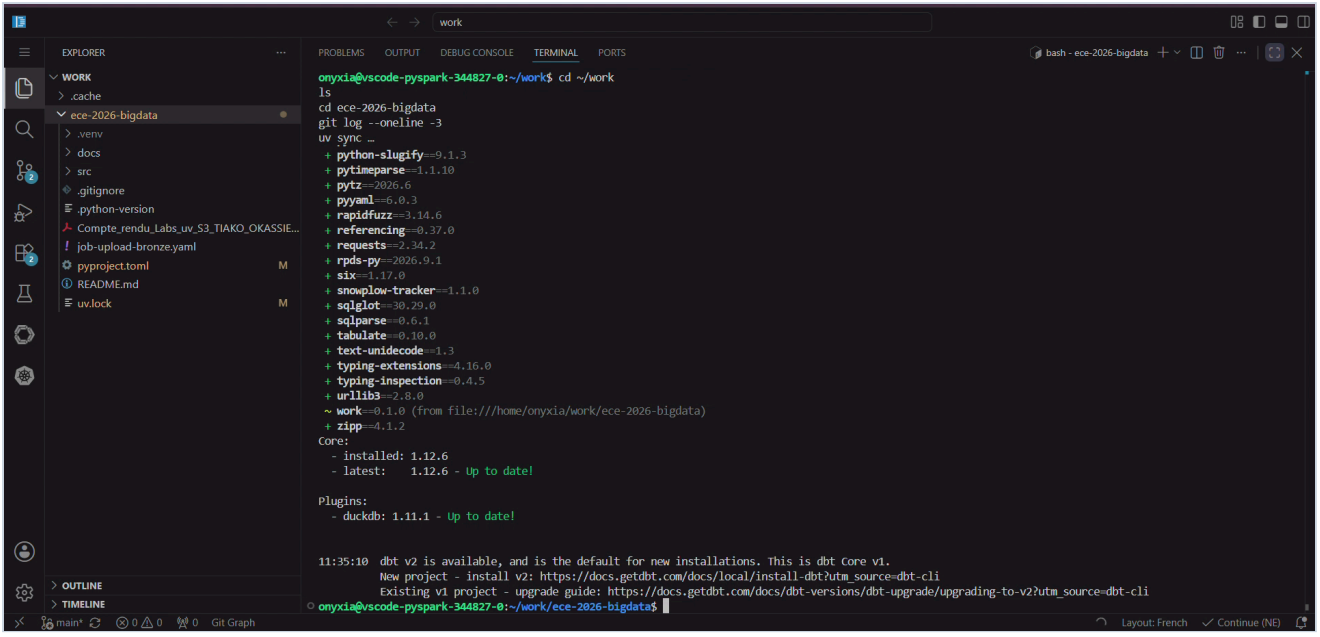


Figure 4 – dbt Core 1.12.6 et plugin duckdb 1.11.1 installés

```
uv run dbt init lab_medallion --skip-profile-setup
cd ~/work/ece-2026-bigdata/lab_medallion/ && rm -rf models/example
```

```
onyxia@vscode-pyspark-344827-0:~/work/ece-2026-bigdata$ uv run dbt init lab_medallion --skip-profile-setup
cd ~/work/ece-2026-bigdata/lab_medallion/
rm -rf models/example
find . -type f | sort
11:44:31  Running with dbt=1.12.6
11:44:32  Creating dbt configuration folder at /home/onyxia/.dbt
11:44:32  Your new dbt project "lab_medallion" was created!
Initialized new project in /home/onyxia/work/ece-2026-bigdata/lab_medallion/lab_medallion

For more information on how to configure the profiles.yml file,
please consult the dbt documentation here:

  https://docs.getdbt.com/docs/configure-your-profile

One more thing:

Need help? Don't hesitate to reach out to us via GitHub issues or on Slack:

  https://community.getdbt.com/

Happy modeling!

./analyses/.gitkeep
./dbt_project.yml
./.gitignore
./macros/.gitkeep
./README.md
./seeds/.gitkeep
./snapshots/.gitkeep
./tests/.gitkeep
```

Figure 5 – Création du projet lab_medallion et arborescence initiale

Le fichier `profiles.yml`, stocké dans le projet (il ne contient aucun secret), indique à dbt d'utiliser la base DuckDB `lab_medallion.duckdb`, avec 4 threads et le fuseau UTC. Les identifiants S3 sont fournis à DuckDB par le secret persistant `s3_onyxia_connection`.

```
11:46:02  Using dbt_project.yml file at /home/onyxia/work/ece-2026-bigdata/lab_medallion/dbt_project.yml
11:46:02  adapter type: duckdb
11:46:02  adapter version: 1.11.1
11:46:02  Configuration:
11:46:02    profiles.yml file [OK found and valid]
11:46:02    dbt_project.yml file [OK found and valid]
11:46:02  Required dependencies:
11:46:02   - git [OK found]

11:46:02  Connection:
11:46:02    database: lab_medallion
11:46:02    schema: main
11:46:02    path: lab_medallion.duckdb
11:46:02    config_options: None
11:46:02    extensions: None
11:46:02    settings: {'TimeZone': 'UTC'}
11:46:02    external_root: .
11:46:02    use_credential_provider: None
11:46:02    attach: None
11:46:02    filesystems: None
11:46:02    remote: None
11:46:02    plugins: None
11:46:02    disable_transactions: False
11:46:02  Registered adapter: duckdb=1.11.1
11:46:03    Connection test: [OK connection ok]

11:46:03  All checks passed!
onyxia@vscode-pyspark-344827-0:~/work/ece-2026-bigdata/lab_medallion$
```

Figure 6 – dbt debug : configuration valide et connexion OK – All checks passed!

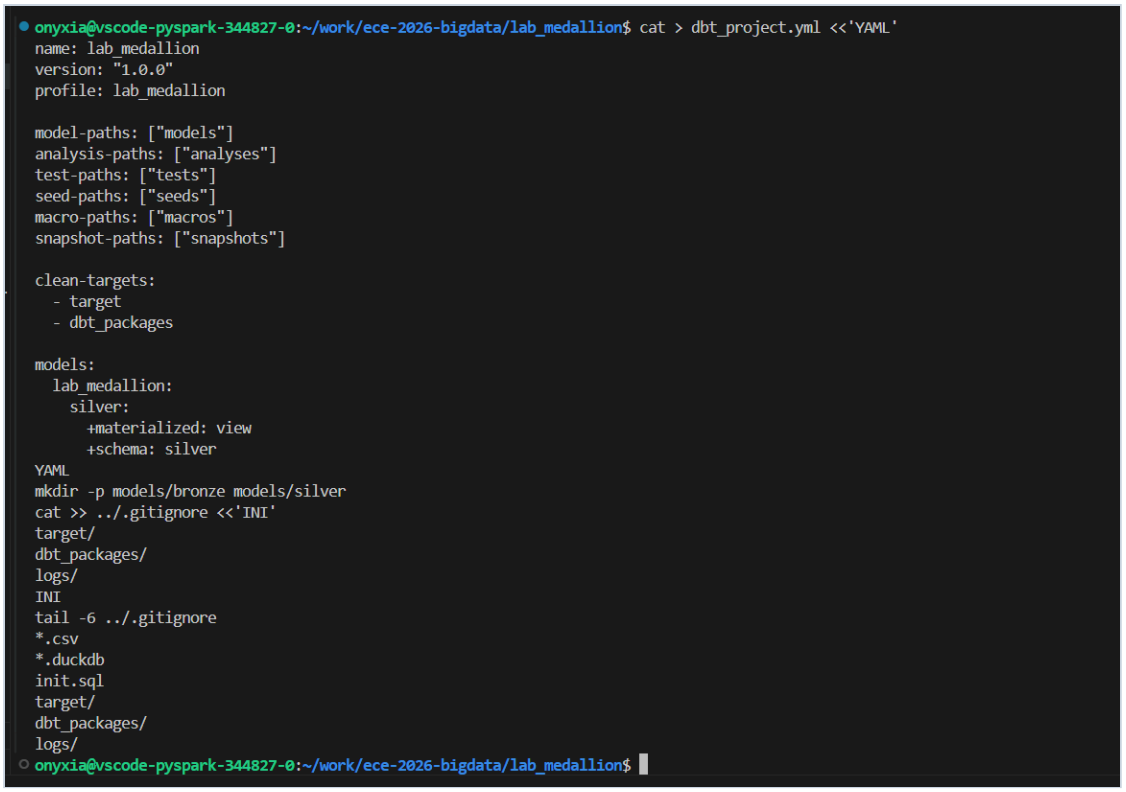


Figure 7 – dbt_project.yml : les modèles du dossier silver sont des vues, dans le schéma silver ; ajout de target/, dbt_packages/ et logs/ au .gitignore

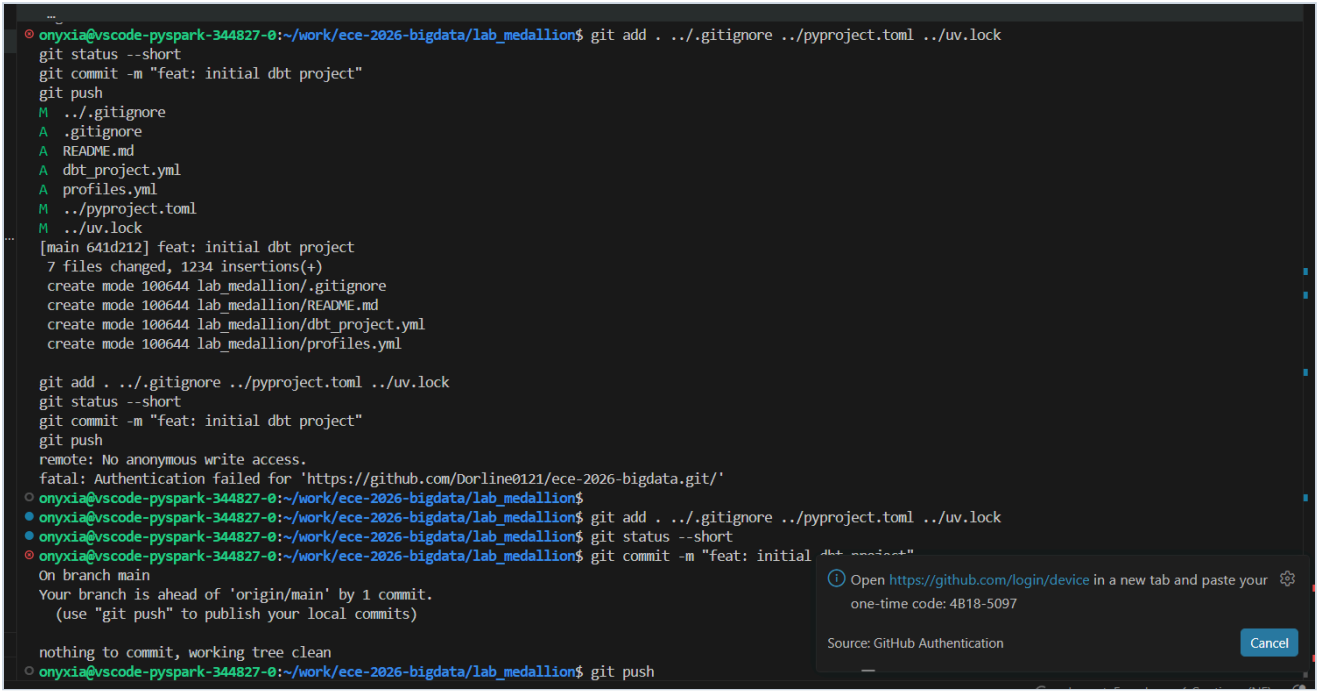


Figure 8 – Premier commit 641d212, mais push refusé faute d'authentification (difficulté ⑤)

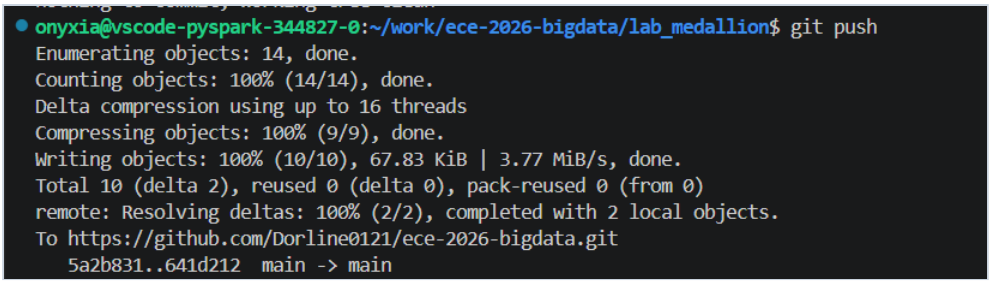


Figure 9 – Push réussi après la connexion GitHub de VS Code

4. Couche bronze : déclaration des sources

dbt n'ingère pas les fichiers bruts : il les déclare comme **sources**. La propriété `external_location`, propre à dbt-duckdb, remplace chaque `{{ source('bronze', ...) }}` par un `read_csv` sur S3. Le nom du bucket est lu dans l'environnement avec `env_var('LAB_BUCKET_NAME')`, ce qui le tient hors du code.

```
sources:
  - name: bronze
    config:
      meta:
        external_location: "read_csv('s3://{{ env_var('LAB_BUCKET_NAME') }}/bronze/{name}.csv', st
rict_mode = false)"
    tables:
      - name: users
      - name: orders
```

```
onyxia@vscode-pyspark-344827-0:~/work/ece-2026-bigdata/lab_medallion$ cat > models/bronze/sources.yml <<'YAML'
sources:
  - name: bronze
    description: Raw datasets uploaded to the S3 bucket by the dataset generators.
    config:
      meta:
        external_location: "read_csv('s3://{{ env_var('LAB_BUCKET_NAME') }}/bronze/{name}.csv', strict_mode = false)"
    tables:
      - name: users
        description: Users generated by `dataset-users`, one row per user.
      - name: orders
        description: Orders generated by `dataset-orders`, one row per order.
YAML
echo "$LAB_BUCKET_NAME"
uv run dbt show -q --limit 3 --inline "select uuid, user_uuid, date, quantity, product from {{ source('bronze', 'orders') }}"
user-d-tiakookassie-ece
| uuid                 | user_uuid            |                 date | quantity | product |
| -------------------- | -------------------- | -------------------- | -------- | ------- |
| 2339ba19-2563-4cc... | bdd640fb-0667-4ad... | 2020-01-01 00:02:... |        4 | drink   |
| b49e04cc-c243-49e... | bdd640fb-0667-4ad... | 2020-01-01 01:28:... |        5 | bread   |
| 41843b03-04dd-405... | bdd640fb-0667-4ad... | 2020-01-01 02:12:... |        2 | donut   |

onyxia@vscode-pyspark-344827-0:~/work/ece-2026-bigdata/lab_medallion$
```

Figure 10 – dbt show : les 3 premières commandes sont lues directement sur S3, identiques au sujet

```
onyxia@vscode-pyspark-344827-0:~/work/ece-2026-bigdata/lab_medallion$ git add models/bronze/sources.yml
git commit -m "feat: add bronze sources definition for users and orders in dbt project"
git push
[main 79545db] feat: add bronze sources definition for users and orders in dbt project
 1 file changed, 11 insertions(+)
 create mode 100644 lab_medallion/models/bronze/sources.yml
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 729 bytes | 364.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Dorline0121/ece-2026-bigdata.git
   641d212..79545db  main -> main
```

Figure 11 – Commit 79545db de la définition des sources

Réponses aux questions

Q1. Les CSV bronze restent intacts. Quel avantage si un bug de transformation est découvert six mois plus tard ?

Le bronze est la **copie brute et immuable** des données reçues. Il suffit de corriger le modèle dbt et de **reconstruire le silver** à partir du bronze d'origine (`dbt build`) : aucune donnée n'est perdue et le recalcul est reproductible. Si les fichiers avaient été modifiés en place, l'erreur serait définitive, car les valeurs d'origine seraient perdues. Le bronze sert aussi de **trace d'audit**, pour comparer le silver à ce qui a réellement été reçu.

Q2. La source relit tout le fichier à chaque requête. Quel format et quelle organisation choisir si les commandes arrivaient toutes les heures pendant des années ?

Un **format colonne, Parquet**, **partitionné par date d'ingestion** (par exemple `bronze/orders/ingest_date=2026-11-05/hour=12/part-0.parquet`), ou une **table Iceberg**. Les requêtes ne lisent alors que les partitions et les colonnes utiles (*partition pruning*, *projection pushdown*). Chaque heure ajoute un fichier sans réécrire les anciens, et dbt peut utiliser un **modèle incrémental** qui ne traite que les nouvelles partitions.

5. Couche silver : modèles `stg_users` et `stg_orders`

Chaque modèle silver est une requête `SELECT` découpée en CTE : une étape lit la source, les suivantes la transforment. Les colonnes sont renommées selon des conventions homogènes (`uuid` devient `user_id`, `mail` devient `email`), puis typées et standardisées.

- **`stg_users`** : l'adresse, sur deux lignes, est découpée en rue, ville, État et code postal. Pour les adresses militaires (`DPO AP 09617`), qui n'ont pas de virgule, la ville vaut `NULL` grâce à `nullif(regexp_extract(...), '')`.
- **`stg_orders`** : identifiants castés en `UUID`, date en `TIMESTAMPTZ`, quantité en `INTEGER`, et dédoublonnage avec `qualify row_number() over (partition by order_id order by ordered_at desc) = 1`. Il protège le modèle contre un rejeu de l'ingestion.

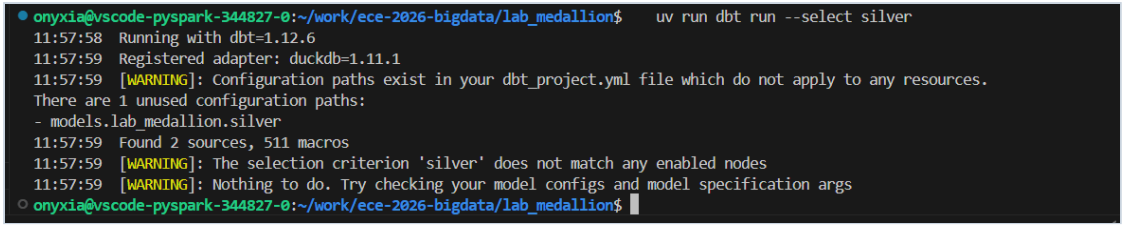


Figure 12 – Premier dbt run : les fichiers SQL n'avaient pas été créés, dbt ne trouve aucun modèle (difficulté ⑤)

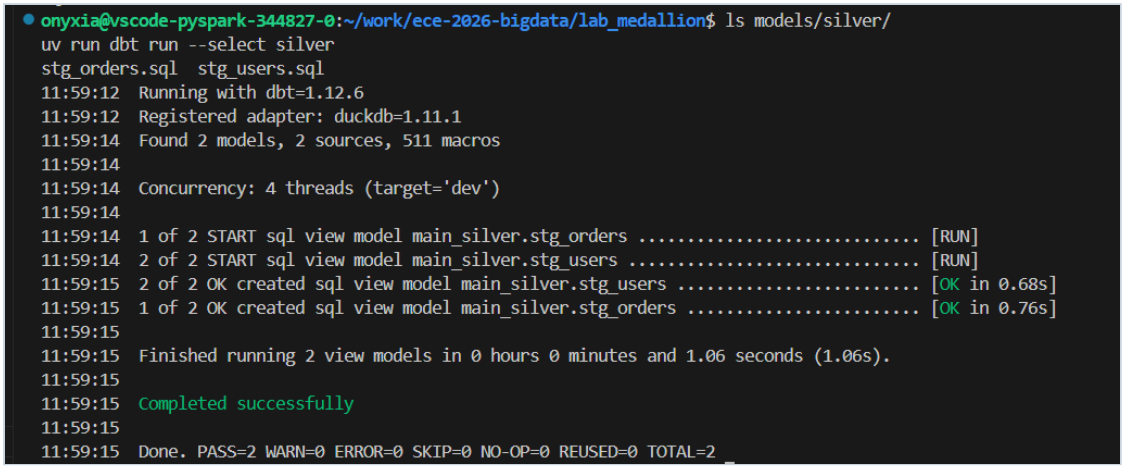


Figure 13 – dbt run --select silver : les deux vues sont créées – PASS=2

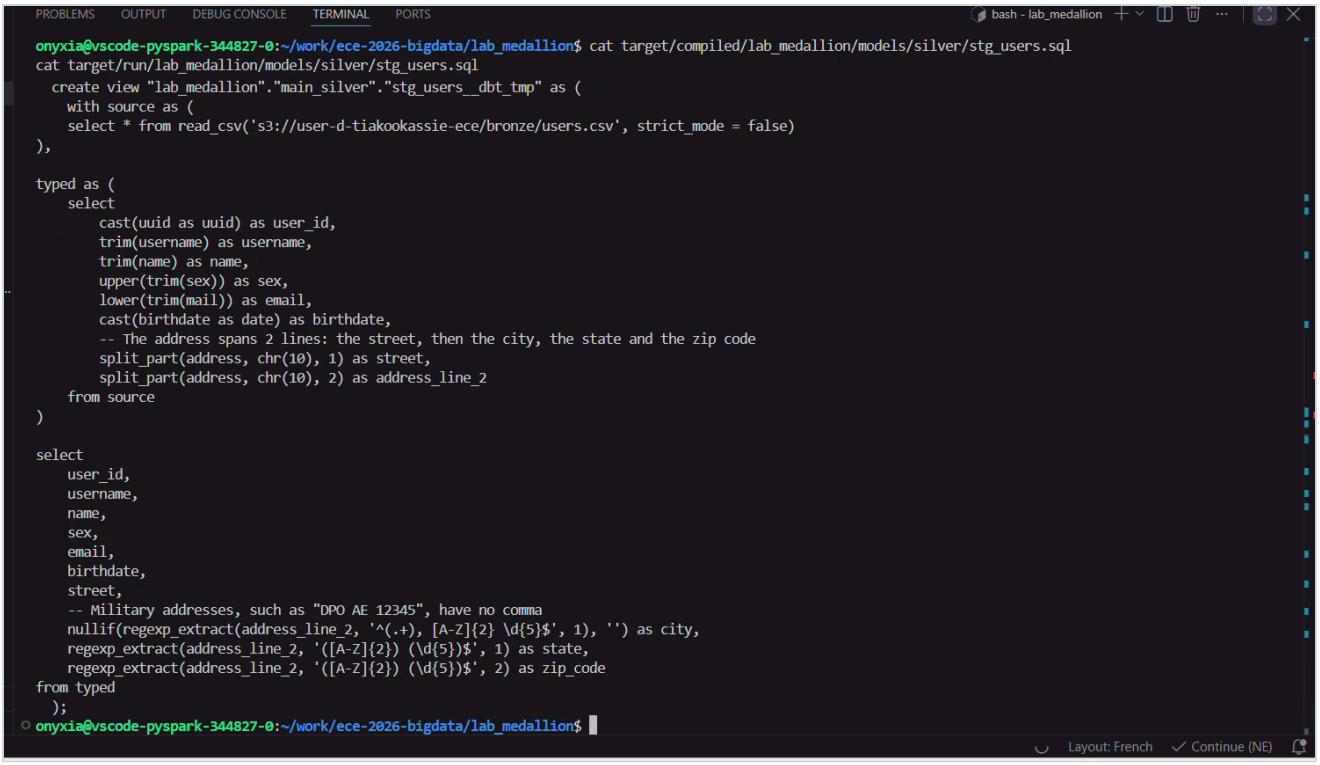


Figure 14 – target/run/ : le SQL compilé, enveloppé dans le DDL create view envoyé à DuckDB

`target/compiled/` contient le `SELECT` où le Jinja est résolu : la source est devenue `read_csv('s3://user-d-tiakookassie-ece/bronze/users.csv', …)`. `target/run/` contient l'instruction réellement exécutée, `create view … as (…)`. Le nom `stg_users__dbt_tmp` montre que dbt crée d'abord une vue temporaire, puis la renomme : la vue en place n'est remplacée que si la création a réussi.

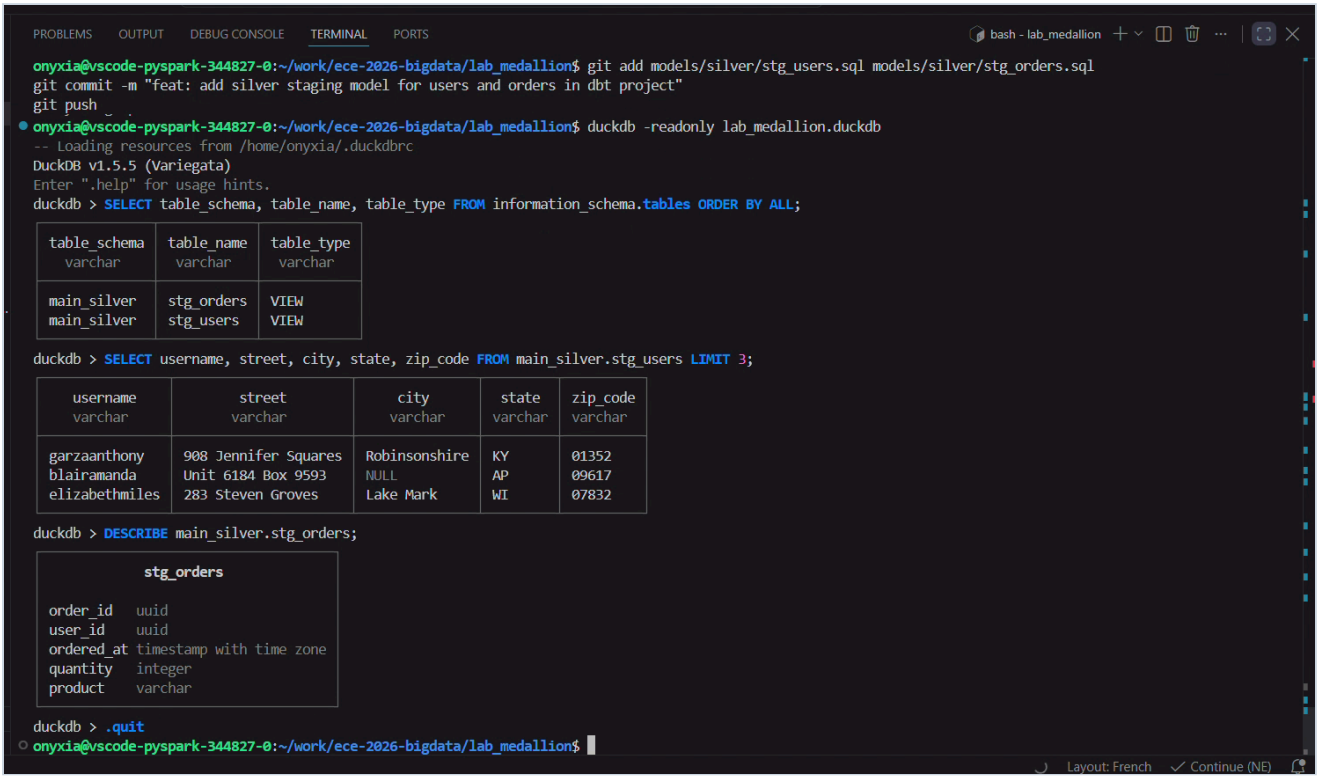


Figure 15 – Inspection en lecture seule : deux vues dans main_silver, adresse découpée (ville NULL pour l'adresse militaire), types UUID, TIMESTAMPTZ et INTEGER

Réponses aux questions

Q3. Les modèles silver sont des vues. Que se passe-t-il quand on interroge une vue, et quand est-elle préférable à une table ?

Une vue ne stocke **que la requête SQL**. À chaque interrogation, DuckDB **réexécute** la transformation : il relit le CSV sur S3, puis refait le typage, le découpage de l'adresse et le dédoublonnage. Le résultat est toujours **à jour** avec le bronze, sans espace disque supplémentaire. Une vue est préférable quand les données sont petites, peu interrogées ou changent souvent, et en développement. Une **table** est préférable dès que la vue est interrogée souvent ou que la transformation est coûteuse (voir l'exercice 5).

Q4. Un UUID est stocké sur 16 octets. Combien d'octets utilise sa représentation VARCHAR ?

Sous forme de texte, un UUID comme `bdd640fb-0667-4ad1-9c80-317fa3b1799d` fait **36 caractères** (32 chiffres hexadécimaux et 4 tirets), soit **36 octets** en ASCII ou UTF-8, plus l'en-tête de chaîne du moteur. C'est plus de deux fois les 16 octets binaires. Le type `UUID` économise de la place, accélère les comparaisons et les jointures, et **valide le format** au moment du cast.

6. Tests de qualité des données

6.1 Seed de référence et tests génériques

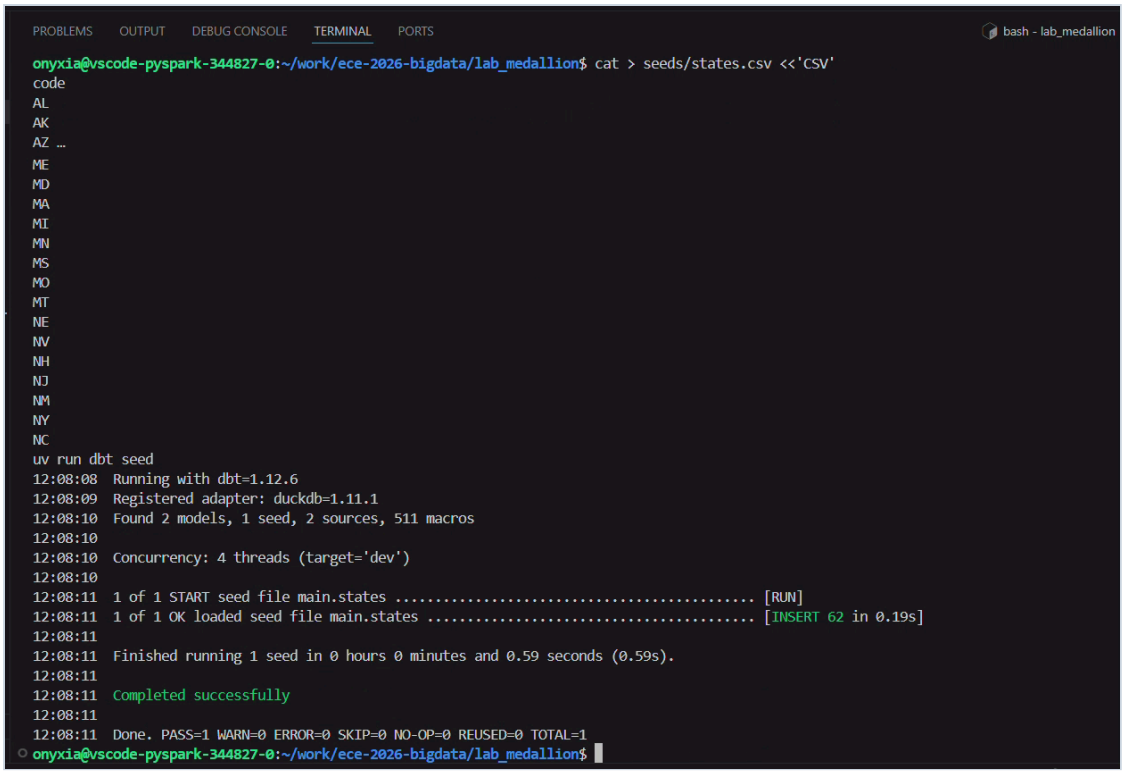


Figure 16 – dbt seed : chargement du référentiel des États (INSERT 62), indispensable au test relationships (difficulté ③)

Les tests génériques sont déclarés en YAML, à côté des modèles. Chacun compile en une requête qui renvoie les lignes violant la règle : le test passe si elle ne renvoie rien.

Modèle	Colonne	Tests
`stg_users`	`user_id`	`unique`, `not_null`
	`email`	`unique`, `not_null`
	`sex`	`accepted_values` (F, M)
	`state`	`not_null`, `relationships` vers `states.code`
`stg_orders`	`order_id`	`unique`, `not_null`
	`user_id`	`not_null`, `relationships` vers `stg_users.user_id`
	`quantity`	`accepted_values` (1 à 5)
	`product`	`accepted_values` (6 produits)

```
12:08:11  Done. PASS=1 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=1
onyxia@vscode-pyspark-344827-0:~/work/ece-2026-bigdata/lab_medallion$    uv run dbt test --select silver
12:10:14  Running with dbt=1.12.6
12:10:15  Registered adapter: duckdb=1.11.1
12:10:16  Found 2 models, 1 seed, 2 sources, 511 macros
12:10:16  [WARNING]: Nothing to do. Try checking your model configs and model specification args
onyxia@vscode-pyspark-344827-0:~/work/ece-2026-bigdata/lab_medallion$
```

Figure 17 – Premier dbt test : silver.yml n'avait pas été créé, aucun test trouvé (difficulté ⑤)

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                                    bash - lab_medallic

onyxia@vscode-pyspark-344827-0:~/work/ece-2026-bigdata/lab_medallion$ ls models/silver/
uv run dbt test --select silver
12:11:30
12:11:30  3 of 13 START test accepted_values_stg_users_sex__F__M ..... [RUN]
12:11:30  1 of 13 START test accepted_values_stg_orders_product__bread__brioche__cookie__croissant__donut__drink  [RUN]
12:11:30  4 of 13 START test not_null_stg_orders_order_id ..... [RUN]
12:11:30  2 of 13 START test accepted_values_stg_orders_quantity__False__1__2__3__4__5 ... [RUN]
12:11:31  3 of 13 PASS accepted_values_stg_users_sex__F__M ..... [PASS in 0.98s]
12:11:31  5 of 13 START test not_null_stg_orders_user_id ..... [RUN]
12:11:31  4 of 13 PASS not_null_stg_orders_order_id ..... [PASS in 1.12s]
12:11:31  6 of 13 START test not_null_stg_users_email ..... [RUN]
12:11:31  2 of 13 PASS accepted_values_stg_orders_quantity__False__1__2__3__4__5 ..... [PASS in 1.18s]
12:11:31  7 of 13 START test not_null_stg_users_state ..... [RUN]
12:11:31  1 of 13 PASS accepted_values_stg_orders_product__bread__brioche__cookie__croissant__donut__drink  [PASS in 1.20s]
12:11:31  8 of 13 START test not_null_stg_users_user_id ..... [RUN]
12:11:32  5 of 13 PASS not_null_stg_orders_user_id ..... [PASS in 0.34s]
12:11:32  9 of 13 START test relationships_stg_orders_user_id__user_id__ref_stg_users_ ... [RUN]
12:11:32  6 of 13 PASS not_null_stg_users_email ..... [PASS in 0.34s]
12:11:32  10 of 13 START test relationships_stg_users_state__code__ref_states_ ..... [RUN]
12:11:32  7 of 13 PASS not_null_stg_users_state ..... [PASS in 0.37s]
12:11:32  11 of 13 START test unique_stg_orders_order_id ..... [RUN]
12:11:32  8 of 13 PASS not_null_stg_users_user_id ..... [PASS in 0.38s]
12:11:32  12 of 13 START test unique_stg_users_email ..... [RUN]
12:11:32  10 of 13 PASS relationships_stg_users_state__code__ref_states_ ..... [PASS in 0.36s]
12:11:32  13 of 13 START test unique_stg_users_user_id ..... [RUN]
12:11:32  12 of 13 PASS unique_stg_users_email ..... [PASS in 0.18s]
12:11:32  9 of 13 PASS relationships_stg_orders_user_id__user_id__ref_stg_users_ ..... [PASS in 0.55s]
12:11:32  13 of 13 PASS unique_stg_users_user_id ..... [PASS in 0.11s]
12:11:32  11 of 13 PASS unique_stg_orders_order_id ..... [PASS in 0.35s]
12:11:32
12:11:32  Finished running 13 data tests in 0 hours 0 minutes and 2.30 seconds (2.30s).
12:11:32
12:11:32  Completed successfully
12:11:32
12:11:32  Done. PASS=13 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=13
onyxia@vscode-pyspark-344827-0:~/work/ece-2026-bigdata/lab_medallion$
```

Figure 18 – Les 13 tests génériques passent – PASS=13

6.2 Tests singuliers et analyse d'un échec

Un test singulier est une requête SQL placée dans `tests/`, qui échoue si elle renvoie des lignes. `assert_orders_have_users` vérifie qu'aucune commande n'est orpheline ; `assert_orders_after_user_birth` vérifie qu'aucun client n'a commandé avant sa naissance.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                   bash - lab_medallion

onyxia@vscode-pyspark-344827-0:~/work/ece-2026-bigdata/lab_medallion$ cat > tests/assert_orders_after_user_birth.sql <<'SQL'
-- A user cannot order before being born: the test fails if this query returns rows
select
    o.order_id,
    o.ordered_at, …
12:14:44  8 of 15 START test not_null_stg_users_email ..................................... [RUN]
12:14:44  5 of 15 PASS assert_orders_have_users ..................................... [PASS in 0.34s]
12:14:44  9 of 15 START test not_null_stg_users_state ..................................... [RUN]
12:14:44  6 of 15 PASS not_null_stg_orders_order_id ..................................... [PASS in 0.32s]
12:14:44  10 of 15 START test not_null_stg_users_user_id ..................................... [RUN]
12:14:44  8 of 15 PASS not_null_stg_users_email ..................................... [PASS in 0.18s]
12:14:44  11 of 15 START test relationships_stg_orders_user_id__user_id__ref_stg_users_ .. [RUN]
12:14:44  9 of 15 PASS not_null_stg_users_state ..................................... [PASS in 0.19s]
12:14:44  12 of 15 START test relationships_stg_users_state__code__ref_states_ ..... [RUN]
12:14:44  7 of 15 PASS not_null_stg_orders_user_id ..................................... [PASS in 0.43s]
12:14:44  13 of 15 START test unique_stg_orders_order_id ..................................... [RUN]
12:14:44  10 of 15 PASS not_null_stg_users_user_id ..................................... [PASS in 0.41s]
12:14:44  14 of 15 START test unique_stg_users_email ..................................... [RUN]
12:14:44  12 of 15 PASS relationships_stg_users_state__code__ref_states_ ..... [PASS in 0.31s]
12:14:44  15 of 15 START test unique_stg_users_user_id ..................................... [RUN]
12:14:45  14 of 15 PASS unique_stg_users_email ..................................... [PASS in 0.13s]
12:14:45  15 of 15 PASS unique_stg_users_user_id ..................................... [PASS in 0.10s]
12:14:45  13 of 15 PASS unique_stg_orders_order_id ..................................... [PASS in 0.36s]
12:14:45  11 of 15 PASS relationships_stg_orders_user_id__user_id__ref_stg_users_ ..... [PASS in 0.59s]
12:14:45
12:14:45  Finished running 15 data tests in 0 hours 0 minutes and 2.45 seconds (2.45s).
12:14:45
12:14:45  Completed with 1 error, 0 partial successes, and 0 warnings:
12:14:45
12:14:45  [ERROR]: in test assert_orders_after_user_birth (tests/assert_orders_after_user_birth.sql)
12:14:45    Got 288 results, configured to fail if != 0
12:14:45
12:14:45    compiled code at target/compiled/lab_medallion/tests/assert_orders_after_user_birth.sql
12:14:45
12:14:45  Done. PASS=14 WARN=0 ERROR=1 SKIP=0 NO-OP=0 REUSED=0 TOTAL=15
onyxia@vscode-pyspark-344827-0:~/work/ece-2026-bigdata/lab_medallion$
```

Figure 19 – assert_orders_after_user_birth échoue : 288 commandes passées avant la naissance du client – PASS=14, ERROR=1

```
onyxia@vscode-pyspark-344827-0:~/work/ece-2026-bigdata/lab_medallion$ uv run dbt test --select assert_orders_after_user_birth --store-failures
duckdb -readonly lab_medallion.duckdb
12:18:15  Running with dbt=1.12.6
12:18:15  Registered adapter: duckdb=1.11.1
12:18:17  Found 2 models, 1 seed, 15 data tests, 2 sources, 511 macros
12:18:17
12:18:17  Concurrency: 4 threads (target='dev')
12:18:17
12:18:17  1 of 1 START test assert_orders_after_user_birth ..... [RUN]
12:18:17  1 of 1 FAIL 288 assert_orders_after_user_birth ..... [FAIL 288 in 0.48s]
12:18:17
12:18:17  Finished running 1 test in 0 hours 0 minutes and 0.92 seconds (0.92s).
12:18:18
12:18:18  Completed with 1 error, 0 partial successes, and 0 warnings:
12:18:18
12:18:18  [ERROR]: in test assert_orders_after_user_birth (tests/assert_orders_after_user_birth.sql)
12:18:18    Got 288 results, configured to fail if != 0
12:18:18
12:18:18    compiled code at target/compiled/lab_medallion/tests/assert_orders_after_user_birth.sql
12:18:18
12:18:18    See test failures:
  ---------------------------------------------------------------------------
  select * from "lab_medallion"."main_dbt_test__audit"."assert_orders_after_user_birth"
  ---------------------------------------------------------------------------
12:18:18
12:18:18  Done. PASS=0 WARN=0 ERROR=1 SKIP=0 NO-OP=0 REUSED=0 TOTAL=1
  Loading resources from /home/onyxia/.duckdbrc
DuckDB v1.5.5 (Variegata)
Enter ".help" for usage hints.
duckdb >
```

Figure 20 – --store-failures enregistre les lignes en échec dans main_dbt_test__audit

```
duckdb > SELECT table_name FROM information_schema.tables WHERE table_schema = 'main_dbt_test__audit';
```

table_name
varchar
accepted_values_stg_orders_de259b370dee6c9743e27a137bfd3ddd
accepted_values_stg_orders_quantity__False__1__2__3__4__5
accepted_values_stg_users_sex__F__M
assert_orders_after_user_birth
assert_orders_have_users
not_null_stg_orders_order_id
not_null_stg_orders_user_id
not_null_stg_users_email
not_null_stg_users_state
not_null_stg_users_user_id
relationships_stg_orders_user_id__user_id__ref_stg_users_
relationships_stg_users_state__code__ref_states_
unique_stg_orders_order_id
unique_stg_users_email
unique_stg_users_user_id

15 rows

Figure 21 – Une table d'échecs par test (15 tables)

```
duckdb > SELECT count(*) AS rows, count(DISTINCT user_id) AS users FROM main_dbt_test__audit.assert_orders_after_user_birth;
```

rows	users
int64	int64
288	5

```
duckdb >
```

Figure 22 – Les 288 lignes concernent seulement 5 utilisateurs

```
duckdb > SELECT * FROM main_dbt_test__audit.assert_orders_after_user_birth LIMIT 5;
```

order_id	ordered_at	user_id	birthdate
uuid	timestamp with time zone	uuid	date
9e44e98c-b359-490e-b030-9aaf85ea16e0	2020-01-20 11:05:42.871698+00	f94d6204-6808-493f-9fed-2c43e256a6dc	2022-11-29
72730e7b-7f9a-4277-9b49-6e6d1688782f	2020-01-20 17:18:57.821499+00	f94d6204-6808-493f-9fed-2c43e256a6dc	2022-11-29
cab24f24-1744-4e3f-936e-c1bd000b1f5e	2020-01-20 19:36:15.131794+00	f94d6204-6808-493f-9fed-2c43e256a6dc	2022-11-29
99966612-9ea8-4552-8475-66bf8f660bbd	2020-01-20 22:28:23.281252+00	f94d6204-6808-493f-9fed-2c43e256a6dc	2022-11-29
0bb81d38-1b37-4f19-b3bc-912f5fb7474e	2020-01-21 01:53:48.757627+00	f94d6204-6808-493f-9fed-2c43e256a6dc	2022-11-29

```
duckdb >
```

Figure 23 – Exemple : un client « né » le 2022-11-29 a commandé dès janvier 2020

Analyse. Les commandes datent de 2020, alors que le générateur (Faker) produit des dates de naissance relatives à la date du jour. Le problème vient donc de **5 dates de naissance fausses**, et non de 288 commandes fausses : chacun de ces 5 clients a beaucoup de commandes, qui héritent toutes de l'erreur. La source est fausse et ne peut pas être corrigée dans le bronze. Une première stratégie consiste à accepter temporairement le problème en le surveillant, avec `severity='warn'` :

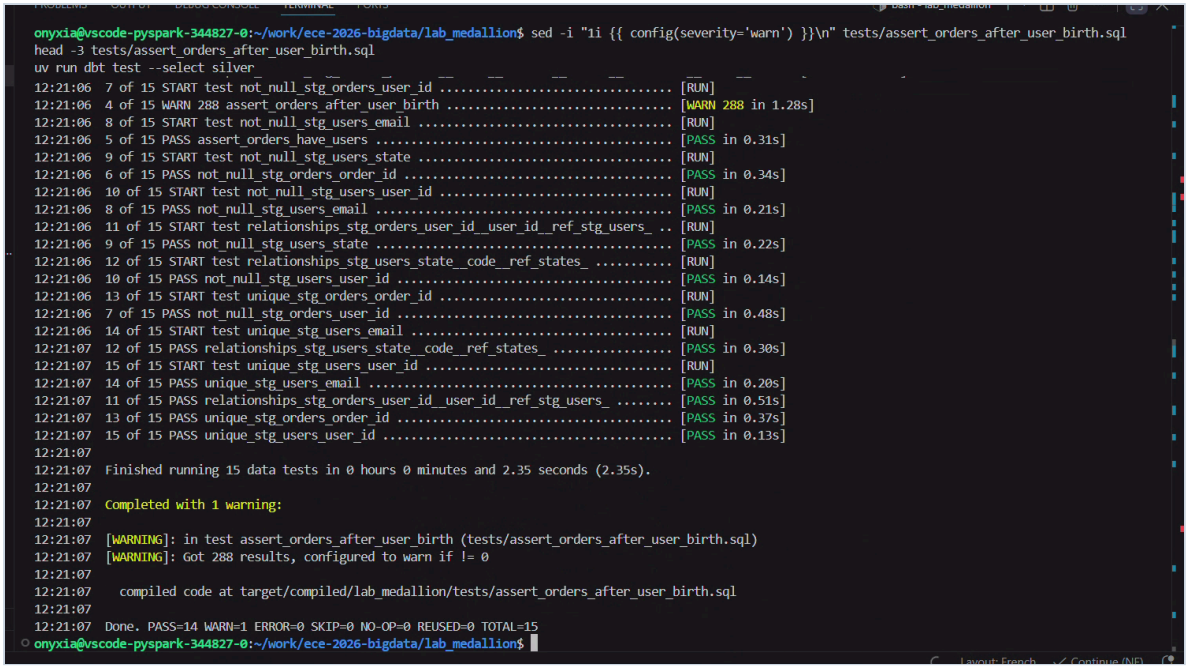


Figure 24 – Le test passe en avertissement : PASS=14, WARN=1, ERROR=0

Réponses aux questions

Q5. Quelle stratégie choisir pour un tableau de bord affichant l'âge des clients ? Pour un tableau de bord des ventes par région ?

- **Âge des clients** : **remplacer les dates invalides par** `NULL` dans le silver, avec un indicateur `birthdate_is_valid` (exercice 3). Un âge négatif fausserait les moyennes et les tranches d'âge ; avec `NULL`, ces clients sont exclus du calcul de l'âge, mais restent dans les autres analyses.
- **Ventes par région** : **garder toutes les lignes** et surveiller le problème (`severity: warn`). La date de naissance n'intervient pas dans ce calcul : rejeter ces 288 commandes ferait sous-estimer les ventes pour une erreur qui ne touche pas les colonnes utilisées.

La bonne stratégie dépend de la colonne fausse et de son usage. Dans tous les cas, il faut signaler le problème au producteur des données.

Q6. Les adresses associent des États à des codes postaux aléatoires (KY 01352). Quel test le détecterait, et quelles données de référence faudrait-il ?

Un **test singulier** qui vérifie que chaque couple (État, code postal) existe dans un référentiel. Les **3 premiers chiffres** d'un ZIP code (le *ZIP prefix*) déterminent l'État : `010–027` pour le Massachusetts, `400–427` pour le Kentucky. Le test joint `stg_users` à un seed d'intervalles de préfixes par État, issu de l'**USPS** (ou des fichiers ZCTA du Census Bureau). `KY 01352` est détecté, car le préfixe `013` correspond au Massachusetts. Ce test est implémenté dans l'exercice 4.

7. Pipeline complet, lineage et documentation

Après suppression de la base, `dbt build` enchaîne, dans l'ordre du graphe de dépendances, le chargement du seed, la création des modèles et l'exécution de leurs tests.

```
rm -f lab_medallion.duckdb
uv run dbt build
```

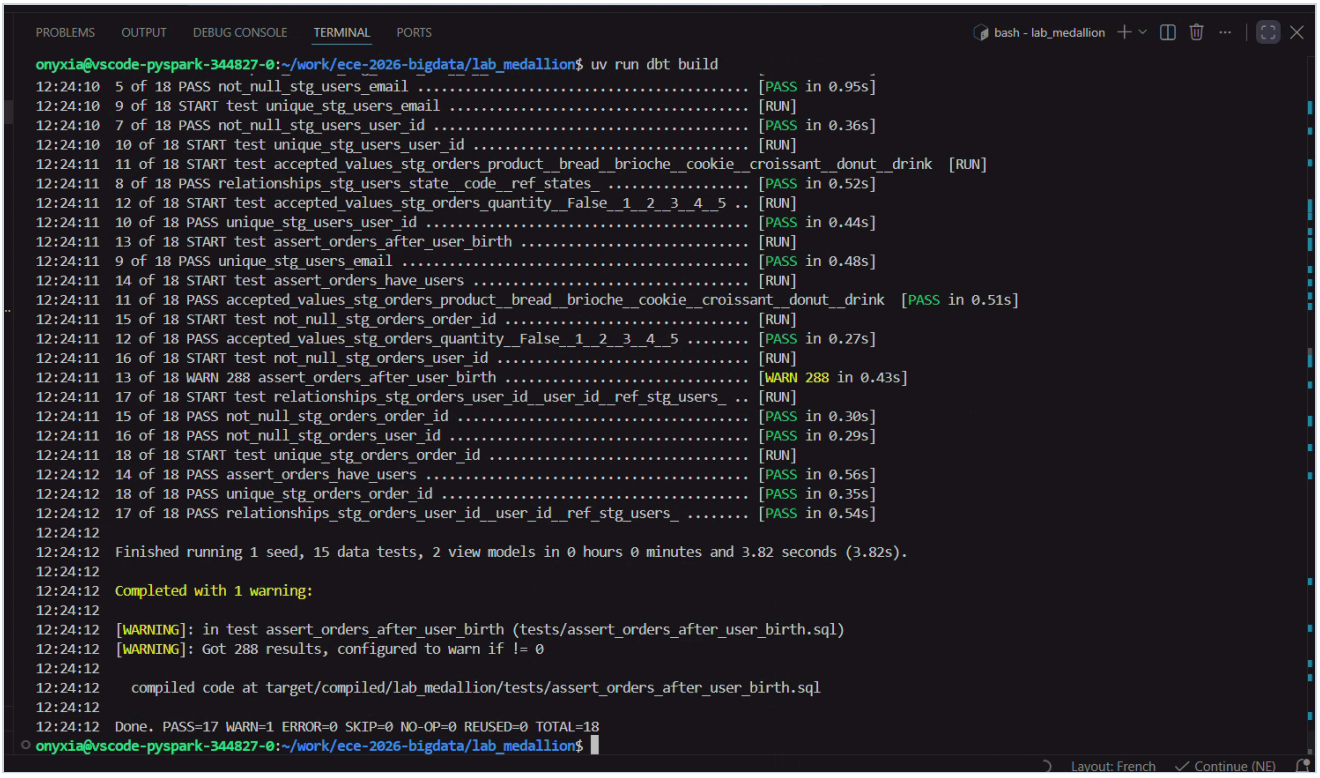


Figure 25 – dbt build depuis une base vide : 1 seed, 2 vues et 15 tests – PASS=17, WARN=1, ERROR=0

dbt connaît les dépendances grâce à `source()`, qui désigne un jeu de données brut externe, et à `ref()`, qui désigne une autre ressource dbt. `dbt ls --select +stg_orders` liste `source:lab_medallion.bronze.orders` puis `lab_medallion.silver.stg_orders` : le `+` signifie « avec tous ses ancêtres ».

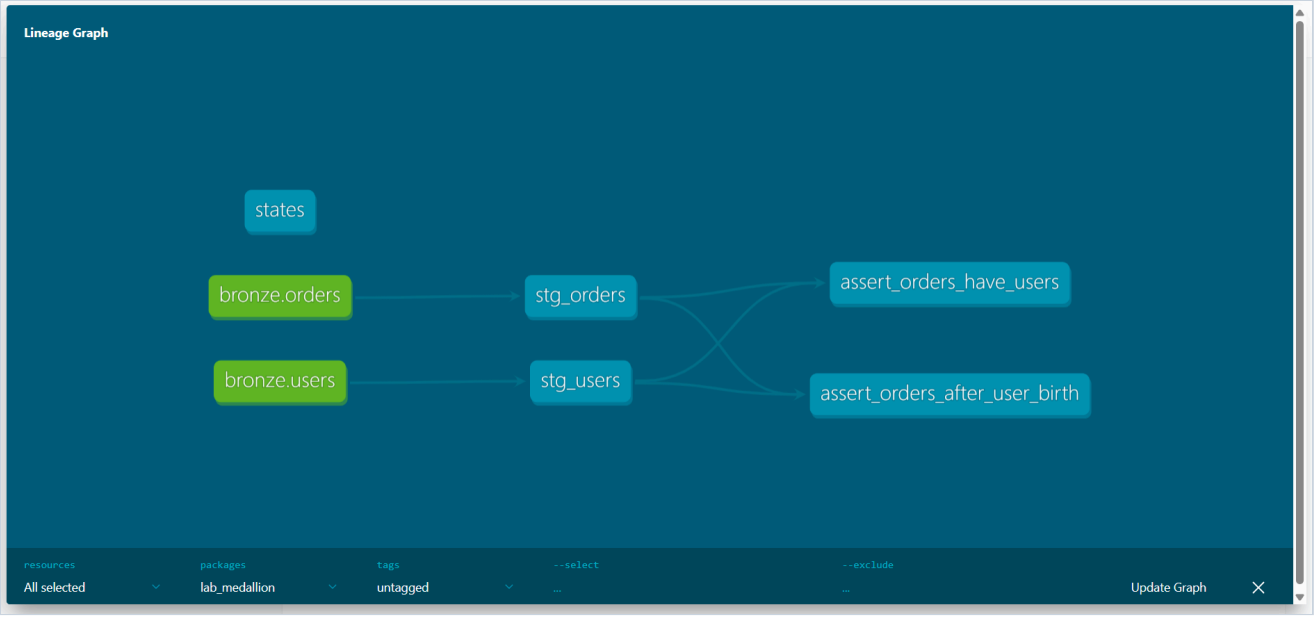


Figure 26 – Graphe de lineage de dbt docs : sources bronze (en vert), modèles silver et tests singuliers

Les tests singuliers dépendent à la fois de `stg_orders` et de `stg_users`, d'où les flèches croisées. Le seed `states` apparaît isolé, car le graphe n'affiche pas les tests génériques comme `relationships`, qui le relient à `stg_users`.

8. Export de la couche silver en Parquet

```
duckdb -readonly lab_medallion.duckdb -c "
COPY (SELECT * FROM main_silver.stg_orders)
TO 's3://$LAB_BUCKET_NAME/silver/dbt/orders.parquet' (FORMAT parquet);"
```

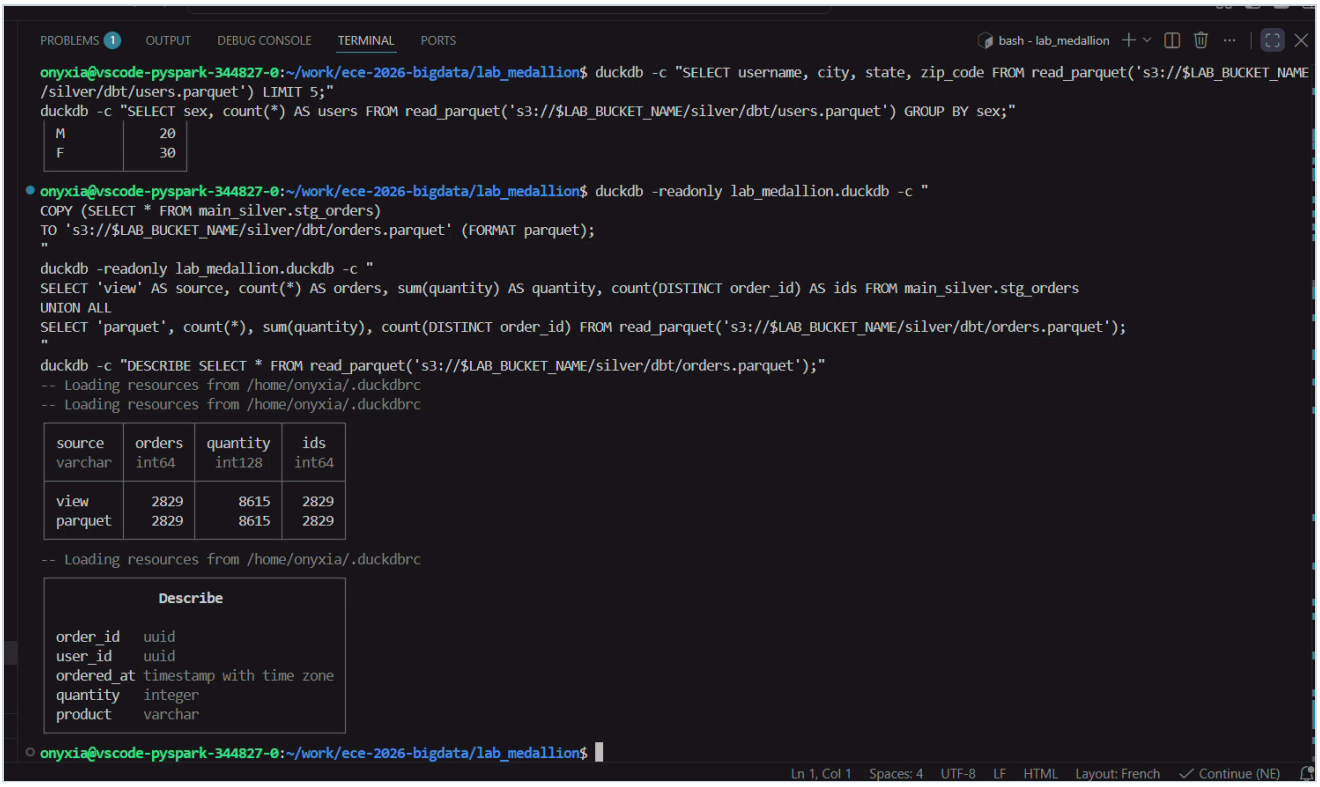


Figure 27 – users.parquet interrogé directement (30 F, 20 M) ; vue et Parquet identiques pour orders (2829 commandes, 8615 articles) ; types conservés

Les deux accès donnent exactement les mêmes résultats : aucune ligne perdue ni dupliquée. Contrairement au CSV, le Parquet **conserve le schéma** (`uuid`, `timestamp with time zone`, `integer`). La **vue** est une matérialisation **logique** : seule la requête est stockée, et elle est recalculée à chaque lecture. Le **Parquet** est une matérialisation **physique** : la transformation est faite une seule fois, à l'export, et les lectures suivantes ne lisent que le résultat.

9. Exercices

Exercice 1 – Colonne `username_normalized`

```
lower(trim(username)) as username_normalized
```

La colonne est documentée dans `silver.yml` et testée avec `not_null` et `unique`. Ces tests garantissent qu'aucun nom d'utilisateur n'est en double, même sans tenir compte de la casse.

Exercice 2 – Validation renforcée des commandes

- `product` **dans la liste attendue** : test générique `accepted_values`, déjà présent.
- **Chaque** `user_id` **existe dans** `stg_users` : test générique `relationships`, confirmé par le test singulier `assert_orders_have_users`.
- `quantity` **strictement positive** : dbt n'a pas de test intégré pour cette règle. J'ai donc écrit un **test générique personnalisé**, réutilisable sur n'importe quelle colonne :

```
{% test positive(model, column_name) %}
select {{ column_name }} from {{ model }} where {{ column_name }} <= 0
{% endtest %}
```

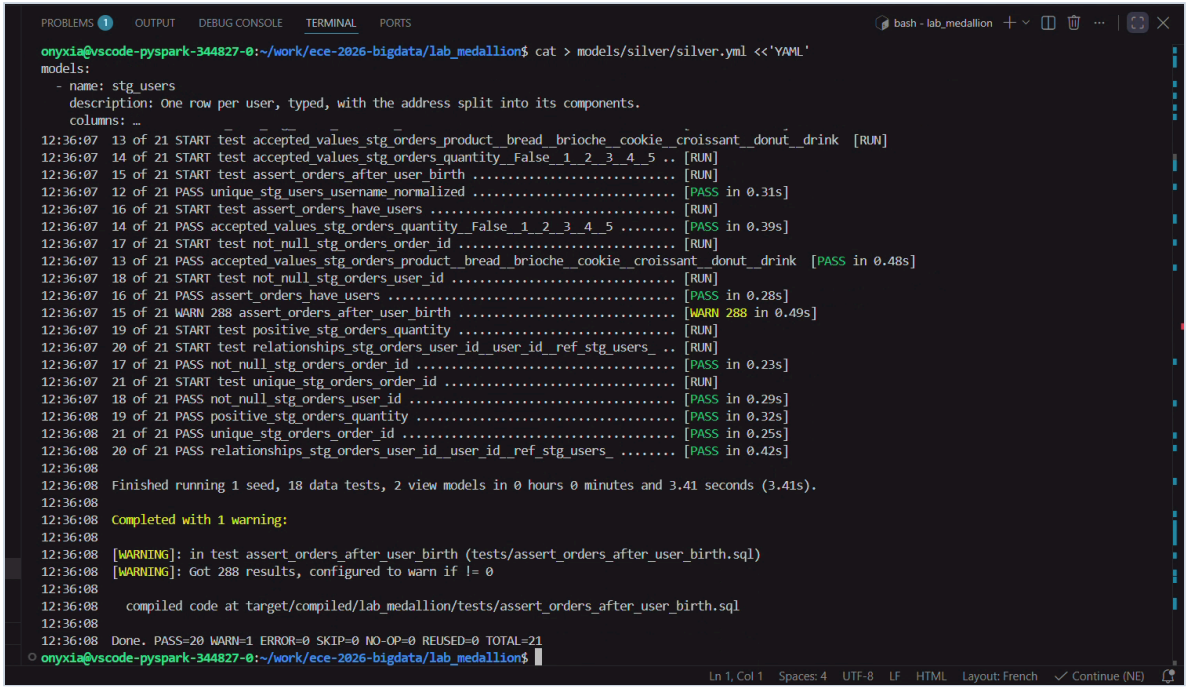


Figure 28 – Exercices 1 et 2 : unique_stg_users_username_normalized et positive_stg_orders_quantity passent – PASS=20, WARN=1

Exercice 3 – Neutraliser les dates de naissance invalides

`stg_users` calcule la première commande de chaque utilisateur (`min(ordered_at)` sur `ref('stg_orders')`). Une date de naissance postérieure à cette première commande est remplacée par `NULL`, et signalée par `birthdate_is_valid = false`. Le test `assert_orders_after_user_birth` redevient bloquant (sans `severity='warn'`).

```
not coalesce(f.first_order_at < cast(t.birthdate as timestamptz), false) as birthdate_is_valid,
case when birthdate_is_valid then birthdate end as birthdate
```

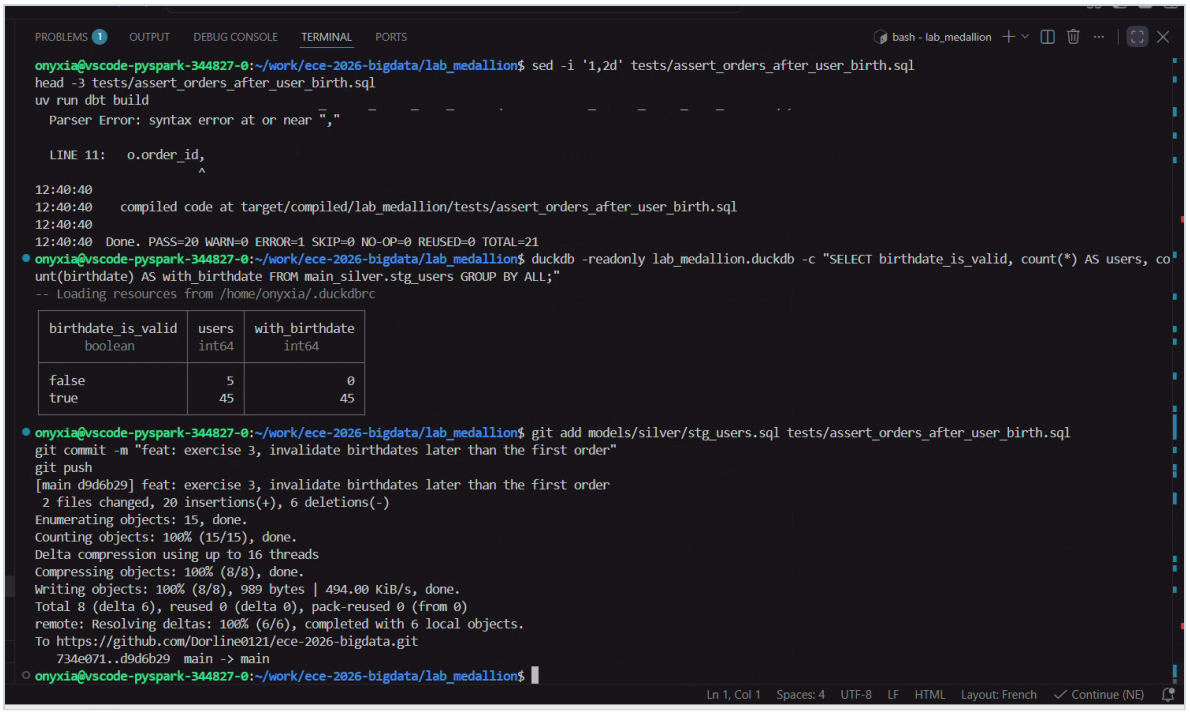


Figure 29 – La suppression de la ligne config avec sed a abîmé le test (Parser Error) ; la vérification montre 45 dates valides et 5 invalides, passées à NULL

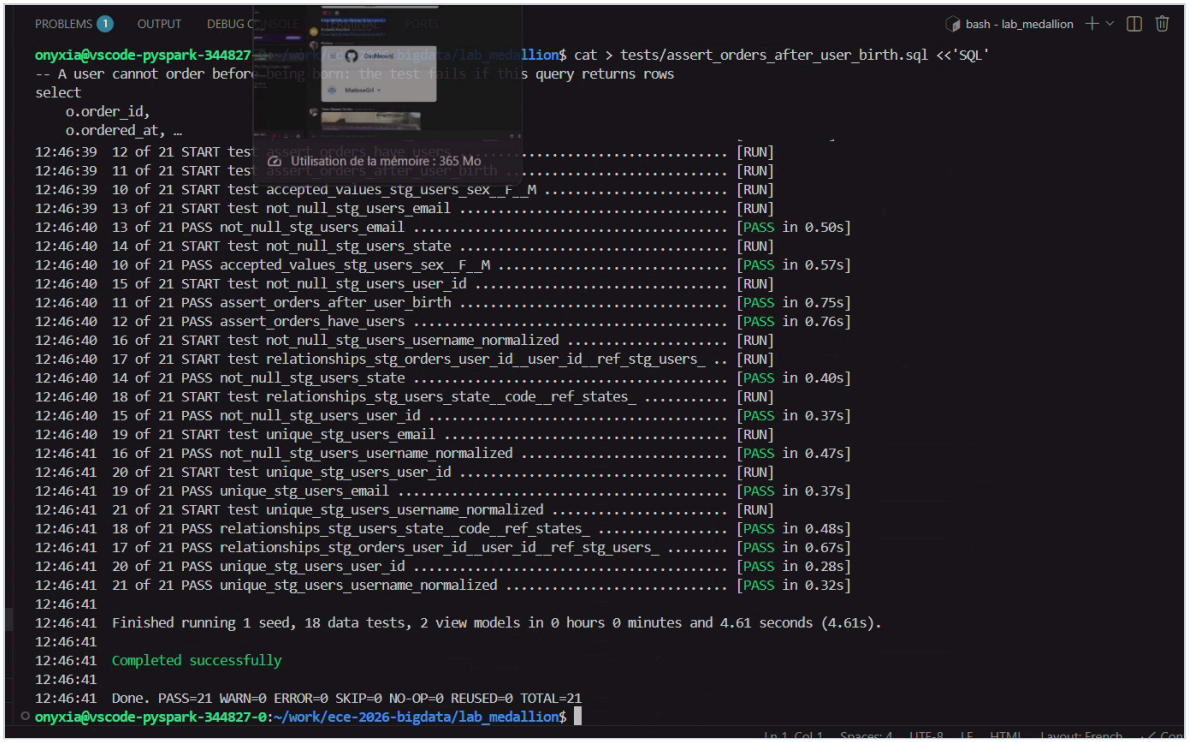


Figure 30 – Test réécrit : assert_orders_after_user_birth passe – PASS=21, WARN=0, ERROR=0

dbt construit désormais `stg_users` **après** `stg_orders`, car le premier dépend du second par un `ref()`. L'ordre est déduit automatiquement du graphe.

Exercice 4 – Détecter les codes postaux incohérents

Un seed `zip_prefixes.csv` (69 intervalles de préfixes à 3 chiffres par État, d'après l'USPS) et un test singulier signalent chaque utilisateur dont le préfixe ne tombe dans aucun intervalle de son État.

```
where not exists (
    select 1 from {{ ref('zip_prefixes') }} z
    where z.state = u.state
      and cast(left(u.zip_code, 3) as integer) between z.zip3_min and z.zip3_max)
```

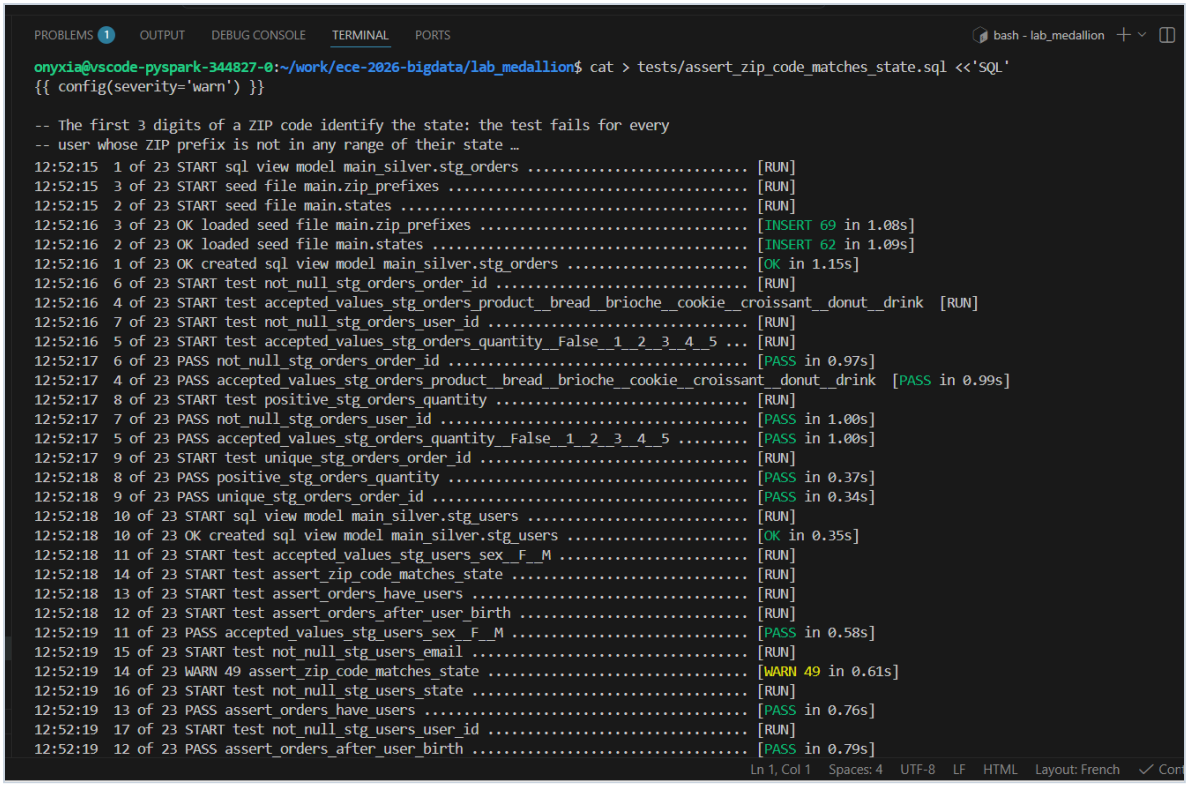


Figure 31 – Seed zip_prefixes chargé (INSERT 69) et test assert_zip_code_matches_state : 49 incohérences

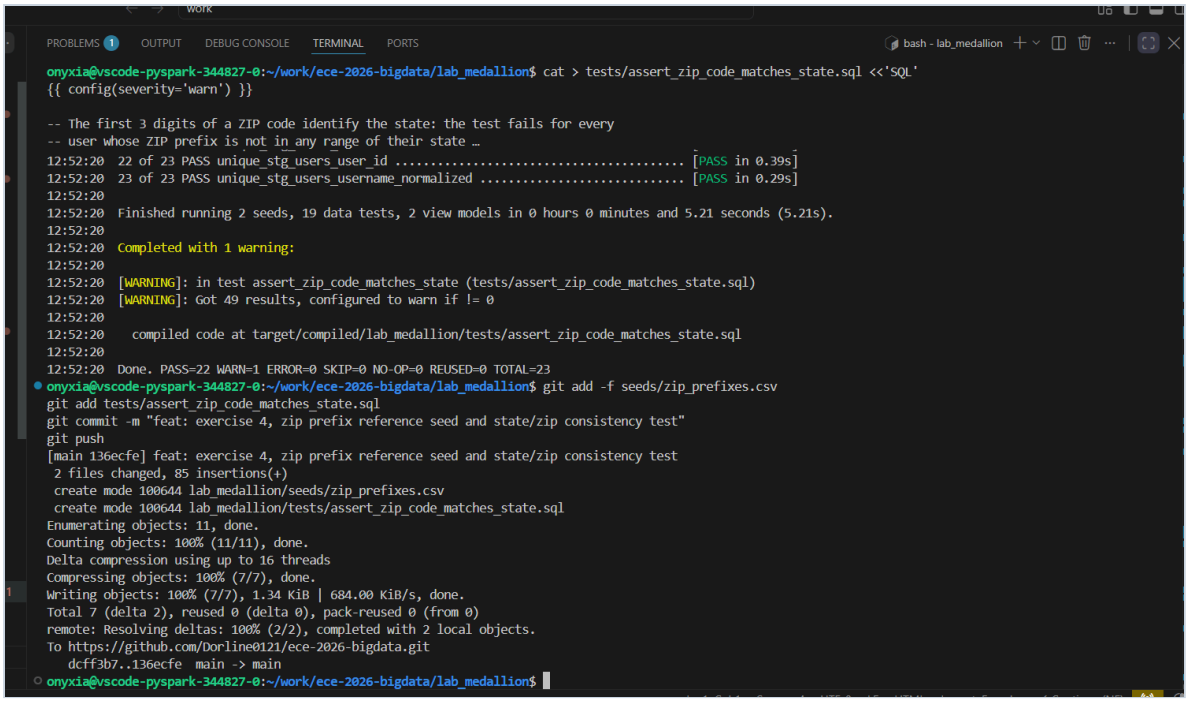


Figure 32 – PASS=22, WARN=1 et commit 136ecfe de l'exercice 4

Résultat. 49 utilisateurs sur 50 ont un code postal incohérent avec leur État. Faker tire l'État et le code postal indépendamment l'un de l'autre ; le seul utilisateur cohérent l'est par hasard.

D'où doivent venir les données de référence ?

D'une **source officielle** : l'USPS, qui attribue les ZIP codes, ou le Census Bureau (ZCTA). Un test ne vaut que ce que vaut son référentiel : pour un usage réel, il faudrait l'importer automatiquement depuis le fichier officiel.

Faut-il le stocker comme seed dbt ?

Oui : il est petit (69 lignes), change rarement et doit être **versionné avec le code**, puisque les tests en dépendent. Pour un référentiel complet de 42 000 codes à 5 chiffres, mis à jour chaque mois, ce serait plutôt une **source** chargée par l'ingestion dans le bronze.

La validation doit-elle se faire dans la couche silver ?

Oui : le silver nettoie et valide. Le test signale le problème, et le modèle peut exposer un indicateur `zip_code_is_valid`, sur le modèle de `birthdate_is_valid`. Il ne faut pas supprimer ces utilisateurs, dont les commandes restent valides : c'est à la couche gold de décider de les exclure, selon l'usage.

Que se passe-t-il quand le référentiel change ?

Il faut mettre à jour le seed, le commiter (Git garde l'historique du référentiel), puis relancer `dbt seed` et `dbt build`. Un code autrefois invalide peut devenir valide, ou l'inverse : les résultats dépendent de la **version du référentiel**, qu'il faut tracer (date de version ou colonne `valid_from`) pour pouvoir expliquer un changement de résultat.

Exercice 5 – Vue ou table

La matérialisation de la couche silver a été passée temporairement de `view` à `table`, puis restaurée.

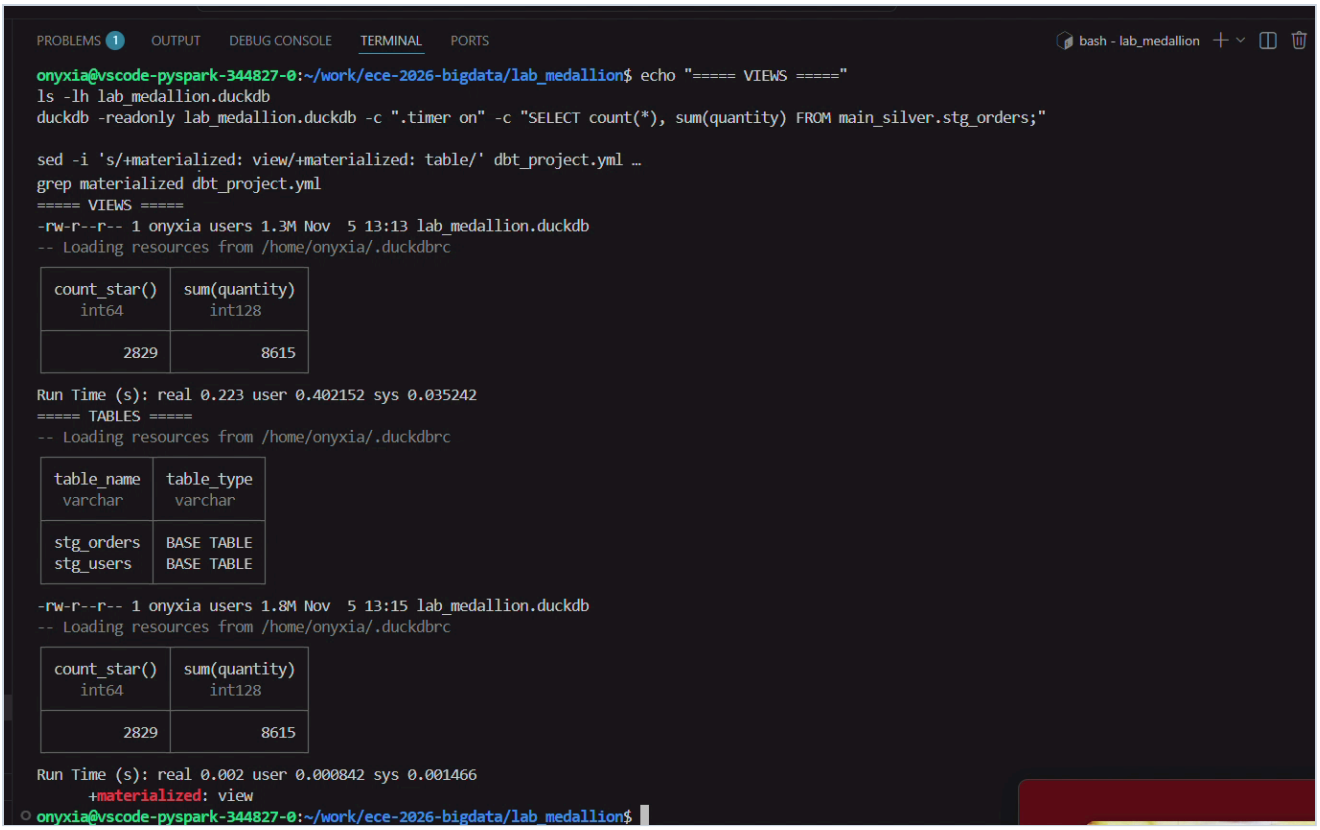


Figure 33 – Mesures avec les vues puis avec les tables : taille de la base, type des relations et temps d'une même requête

	Vues (`view`)	Tables (`table`)
Type dans `information_schema`	`VIEW`	`BASE TABLE`
Taille de `lab_medallion.duckdb`	1,3 Mo	1,8 Mo
Temps de `count(*), sum(quantity)` sur `stg_orders`	0,223 s	**0,002 s**
Résultat	2829 / 8615	2829 / 8615

Où sont stockées les données ?

Avec une **vue**, la base ne contient que la requête ; les données restent dans le CSV sur S3. Avec une **table**, les données transformées sont copiées dans `lab_medallion.duckdb`, qui passe de 1,3 à 1,8 Mo.

Quand la transformation est-elle exécutée ?

Pour une vue, **à chaque requête**. Pour une table, **une seule fois**, au moment de `dbt run` ou `dbt build`.

Que se passe-t-il quand le CSV sous-jacent change ?

La vue reflète le changement **immédiatement**. La table reste figée sur l'état du dernier build : elle est **périmée** jusqu'à la prochaine exécution de dbt.

Quelle approche consomme le plus de stockage ?

La **table**, qui duplique les données : environ 0,5 Mo de plus ici, une quantité qui croît avec le volume.

Quelle approche évite de relire les fichiers sources à chaque requête ?

La **table** : la requête est environ **100 fois plus rapide** (0,002 s contre 0,223 s), car DuckDB lit ses propres données en local au lieu de télécharger et d'analyser le CSV sur S3 et de refaire toute la transformation.

La configuration d'origine (`+materialized: view`) a été restaurée, et le dernier `dbt build` confirme `2 view models`.

10. Synthèse des résultats

Étape	Résultat
Source bronze	2 sources lues directement sur S3 par `read_csv`
Modèles silver	2 vues : `stg_users` (50 lignes) et `stg_orders` (2829 lignes)
Tests génériques	13 tests, tous en `PASS`
Test singulier « avant naissance »	288 commandes en échec, pour 5 utilisateurs
Pipeline initial (`dbt build`)	`PASS=17 WARN=1 ERROR=0`
Export Parquet	Vue et Parquet identiques (2829 / 8615), types conservés
Exercices 1 et 2	`username_normalized` et test générique `positive` : `PASS=20 WARN=1`
Exercice 3	5 dates invalides passées à `NULL` : `PASS=21 WARN=0 ERROR=0`
Exercice 4	49 couples État / code postal incohérents détectés (`WARN`)
Exercice 5	Table environ 100 fois plus rapide qu'une vue, pour 0,5 Mo de stockage en plus

La couche bronze et les fichiers `silver/dbt/*.parquet` sont conservés sur S3 : le lab Spark suivant reconstruit la même couche silver et la compare à celle de dbt.

11. Conclusion

Ce lab montre comment dbt structure une transformation de données comme un projet logiciel : les sources sont déclarées, chaque modèle est une requête versionnée, les dépendances sont déduites de `source()` et `ref()`, et la qualité est vérifiée par des tests exécutés à chaque build. DuckDB fournit un moteur suffisant pour lire directement les fichiers bruts sur S3, sans serveur.

Les tests ont surtout montré leur valeur en révélant des problèmes réels de la source : des dates de naissance incohérentes et des codes postaux sans rapport avec l'État. La couche bronze n'étant jamais modifiée, ces problèmes se traitent dans le silver, en neutralisant les valeurs fausses et en les signalant par un indicateur, plutôt qu'en supprimant des lignes encore utiles. Enfin, le choix entre vue et table illustre le compromis central entre fraîcheur des données, coût de stockage et performance des requêtes.